

ClubGear

A Campus Club Equipment Booking and Management System

Project Report
Java Programming — Flipped Classroom Evaluation

Student name	Suyash Singh
Registration number	25BAI11183
Programme	B.Tech, Computer Science and Engineering
Year / Semester	Second year
Course	Java Programming
Faculty	DR. Vipin Jain
Submission date	18 September 2026
Repository	https://github.com/suyash25bai11183-ui/Java-Project

Table of Contents

1	Introduction
2	Problem Statement
3	Functional Requirements
4	Non-Functional Requirements
5	System Architecture
6	Design Diagrams (use case, workflow, sequence, class, ER)
7	Design Decisions and Rationale
8	Implementation Details
9	Screenshots and Results
10	Testing Approach
11	Challenges Faced
12	Learnings and Key Takeaways
13	Future Enhancements
14	References

Abstract

ClubGear is a console based Java application that manages the equipment owned by college clubs — cameras, microphones, projectors, tripods and lighting panels — and the bookings made against that equipment by club members. The system enforces twelve business rules, including the prevention of overlapping bookings, the blocking of damaged equipment, and the automatic detection of overdue returns. It is written entirely in core Java 17 with no frameworks, no database and no external libraries, and stores its data in five plain CSV files. The implementation consists of eleven classes across three layers, supported by a self-written suite of 55 automated validation tests and a 22-case manual test plan.

Project status. The project is complete and working. It compiles with `javac --release 17` with no errors and no warnings (verified with `-Xlint:all`), all 55 automated tests pass, and every business rule listed in section 3 has been verified against the running program.

1. Introduction

Most college clubs own equipment that members share: the photography club has cameras and tripods, the coding club has projectors and microphones, the cultural committee has speakers and lighting. This equipment is expensive, it is used by dozens of people across the year, and it is almost always tracked on paper.

ClubGear replaces that paper register with a menu driven Java program. Every club, member, equipment item, booking and maintenance entry receives a unique generated ID. When a member requests equipment, the program checks the request against the existing bookings and refuses anything that would cause a conflict, naming the exact rule that was broken. When equipment is returned, its condition is recorded, and a damaged item is locked out of the booking system until it has been through maintenance.

1.1 Purpose of this report

This report documents the problem, the requirements, the design, the implementation and the testing of ClubGear. It is written so that a reader who has never seen the source code can understand what the system does, how it is built, and why each significant decision was made.

1.2 Scope of the project

In scope	Out of scope
Menu driven console application in Java 17 Clubs, members, equipment, bookings, maintenance Business rule validation on every operation On-screen reports for the coordinator Saving and loading data in CSV text files Built-in sample data for demonstration	Web or mobile interfaces MySQL, Spring Boot, Hibernate, any framework Login, passwords and access control Fines, payments and accounting Networking, multithreading, concurrent users Any external library or build tool

1.3 Technologies used

Item	Detail
Language	Java 17 (compiled with <code>--release 17</code> ; runs on 17 and above)
Input handling	<code>java.util.Scanner</code> , wrapped by a validation class
Collections	<code>ArrayList</code> for records, <code>HashSet</code> for duplicate emails, <code>HashMap</code> for report counting
Dates	<code>java.time.LocalDate</code> , <code>java.time.LocalDateTime</code>
File handling	<code>BufferedReader</code> , <code>BufferedWriter</code> , <code>FileReader</code> , <code>FileWriter</code> , <code>File</code>
Storage format	Five comma separated text files in a <code>data</code> folder
Testing	Self-written validation test runner (no JUnit) plus a manual test plan
Version control	Git and GitHub
Build	Plain <code>javac</code> ; no Maven, no Gradle

1.4 Target users

User	How they use ClubGear
Club coordinator	Adds the club and its equipment, approves and cancels bookings, issues and receives equipment, reads the reports
Club member	Checks which equipment is free, requests a booking with a purpose, returns the item on time
Equipment in-charge	Records the condition of returned items, reports damage, manages maintenance
Department office	Uses the reports to see how much equipment each club owns and how often it is used

2. Problem Statement

College clubs share expensive equipment among many members, and the borrowing process is managed manually through a register in the club room or through informal messages. This manual method fails in five specific ways:

Problem	Consequence
Two members request the same item for the same date	The clash is discovered on the morning of the event, when it is too late to arrange an alternative
No reliable record of who is currently holding an item	Items go missing and nobody can be held responsible
Late returns are not noticed	The next event is disrupted because the equipment has not come back
Damage is never written down	A broken item is issued again and the next member is blamed for damage they did not cause
No usage information exists	The coordinator cannot answer "which item is used most" or "what should we buy next year"

Statement. A small, reliable software system is required that records clubs, members and equipment, processes booking requests through a defined life cycle, automatically rejects requests that violate the club's borrowing rules, records the condition of returned items, and keeps all of this information between runs of the program — without requiring a database server, an internet connection or any software beyond a Java runtime.

2.1 Objectives

1. Maintain a digital record of clubs, members and club-owned equipment.
2. Allow members to book equipment for a date range with a stated purpose.
3. Prevent double bookings by checking date overlaps automatically.
4. Prevent the booking of damaged or under-maintenance equipment.
5. Manage a booking through its full life cycle: requested, approved, active, returned, cancelled, overdue.
6. Record the condition of equipment after return and keep a maintenance history.
7. Detect overdue bookings using the current system date.
8. Generate reports that help a coordinator make decisions.
9. Store all data in plain text files so the information survives between runs.
10. Demonstrate the practical use of core Java: classes, objects, encapsulation, collections, enums, exception handling, date classes and file handling.

2.2 The twelve business rules

These rules are the heart of the system. Each one is enforced at a single identified place in the code, and each one is covered by at least one test case.

#	Rule	Enforced in
1	Club names must not be duplicated	<code>addClub()</code> via <code>findClubByName()</code>
2	Member emails must not be duplicated	<code>registerMember()</code> via <code>HashSet</code>
3	Equipment quantity must be greater than zero	<code>readInt(prompt, 1, 500)</code> and <code>setQuantity()</code>
4	Damaged or under-maintenance equipment cannot be booked	<code>Equipment.isBookable()</code>
5	A member cannot book the same item for overlapping dates	<code>findClashingBooking()</code>
6	Two bookings for the same item cannot overlap	<code>Booking.overlapsWith()</code>
7	Return date cannot be earlier than the start date	<code>createBooking()</code> , <code>returnEquipment()</code>
8	Only approved bookings can become active	<code>issueEquipment()</code>
9	Only active (or overdue) bookings can be returned	<code>returnEquipment()</code>
10	Only returned or cancelled bookings release the equipment	<code>releaseEquipment()</code>
11	Overdue bookings are detected using the current date	<code>updateOverdueBookings()</code>
12	Invalid menu input must not crash the program	<code>InputValidator.readInt()</code>

3. Functional Requirements

The system is built from seven functional modules. The course guidelines require a minimum of three; ClubGear has seven, because a booking life cycle is only meaningful when equipment, members and maintenance are all present.

#	Module	Capabilities
1	Club management	Add clubs with generated IDs, view all, search by name or ID, reject duplicate names
2	Member management	Register members with generated IDs, validate email format, reject duplicate emails, view and search
3	Equipment management	Add equipment with generated IDs, store quantity, condition, owner club and status; search by name or category; update condition
4	Equipment booking	Select member, equipment and club; enter dates and purpose; check availability; detect clashes; print a confirmation slip
5	Booking status management	Approve, cancel, issue, return; detect overdue bookings; refuse invalid status transitions
6	Return and maintenance	Record return date and condition, log damage notes, send for maintenance, mark repaired, keep a history
7	Reports	Totals, availability counts, overdue list with late days, most booked item, club-wise usage

3.1 Detailed requirements

ID	Requirement
FR1	The system shall add a club with a unique auto generated ID and reject duplicate club names.
FR2	The system shall display all clubs and search clubs by name or ID.
FR3	The system shall register a member with a unique auto generated ID and reject a duplicate email.
FR4	The system shall display all members and search members by name or ID.
FR5	The system shall add equipment with a unique auto generated ID and accept only a quantity greater than zero.
FR6	The system shall display equipment, search it by name or category, and update its condition.
FR7	The system shall mark equipment as AVAILABLE, BOOKED, DAMAGED or UNDER_MAINTENANCE.
FR8	The system shall create a booking containing member, equipment, club, start date, expected return date and purpose.
FR9	The system shall reject a booking whose return date is earlier than its start date.
FR10	The system shall reject a booking that overlaps an existing live booking of the same equipment.
FR11	The system shall reject a booking of damaged or under-maintenance equipment.
FR12	The system shall display a booking confirmation after a successful booking.
FR13	The system shall allow a requested booking to be approved or cancelled.
FR14	The system shall allow only an approved booking to be issued and become active.
FR15	The system shall allow only an issued booking (active or overdue) to be returned.
FR16	The system shall record the actual return date, the condition after return and damage notes.
FR17	The system shall release the equipment when a booking is returned or cancelled.
FR18	The system shall detect overdue bookings by comparing the due date with the current date.
FR19	The system shall allow equipment to be sent for maintenance and later marked as repaired.
FR20	The system shall keep a maintenance history for every item.
FR21	The system shall display reports covering totals, availability, overdue bookings, most booked equipment and club-wise usage.
FR22	The system shall save all data to CSV files and load it back on request or at start-up.
FR23	The system shall continue running normally when an invalid menu choice, number or date is entered.

3.2 Input and output structure

Direction	Structure
Input	Numbered menu choices (1-16); IDs typed as text such as MEM003 (case insensitive); dates typed as YYYY-MM-DD ; free text for names, purposes and notes; y / n confirmations. Every input is read as a full line and parsed

	afterwards.
Output	Column-aligned tables for lists; a boxed confirmation slip for a new booking; a reports page; specific error messages that name the rule that was broken; five CSV files written to disk.

4. Non-Functional Requirements

The guidelines ask for at least four non-functional requirements. Seven are specified below, each with the way it is actually achieved in the code rather than as a general claim.

ID	Quality	Requirement and how it is met
NFR1	Usability	The menu is numbered and always visible; every error message names the specific problem (for example <i>"Error: booking BKG005 is CANCELLED. Only APPROVED bookings can be issued."</i>) rather than a generic failure. Lists are shown before any prompt that asks for an ID, so the user never has to remember one.
NFR2	Reliability	No user input can terminate the program. Numbers are range checked, dates are parsed inside a <code>try / catch</code> for <code>DateTimeParseException</code> , and every invalid entry causes a re-prompt. Verified by test cases TC-11 and TC-19.
NFR3	Error handling strategy	Three levels: (a) prevention — validated input; (b) rule checks that refuse the operation and return to the menu without changing any data; (c) recovery — a damaged CSV line returns <code>null</code> from <code>fromCsvLine()</code> and is skipped with a counted warning instead of aborting the load.
NFR4	Maintainability	Three layers with a single responsibility each; eleven classes; methods are short (the longest is under 60 lines); beginner-oriented comments explain each business rule at the point where it is enforced; the CSV format is documented in the class comment of <code>FileStorage</code> .
NFR5	Performance	All data is held in memory, so searches are linear over a few hundred records and respond instantly. Duplicate email checking is $O(1)$ through a <code>HashSet</code> , and report counting is a single pass with a <code>HashMap</code> instead of nested loops.
NFR6	Portability	Runs on Windows, Linux and macOS with only a JDK 17 installation. No native libraries, no absolute paths: the data folder is created relative to the working directory.
NFR7	Data safety	A missing data folder or file is treated as "nothing saved yet", not as an error. Loading clears the in-memory lists first, so loading twice never duplicates records. Commas are removed from user text at entry so a stored line can never be split into the wrong number of columns.

On security. Security is deliberately *not* claimed as a non-functional requirement. There is no login, no password and no encryption, so any claim of security would be false. The honest position is that ClubGear is a single-user offline tool whose data file is as protected as the folder it sits in, and adding role-based login is listed as the first future enhancement in section 13.

5. System Architecture

ClubGear uses a three-layer architecture. Each layer communicates only with the layer below it, which keeps the responsibilities separate and makes the domain classes testable without any keyboard input.

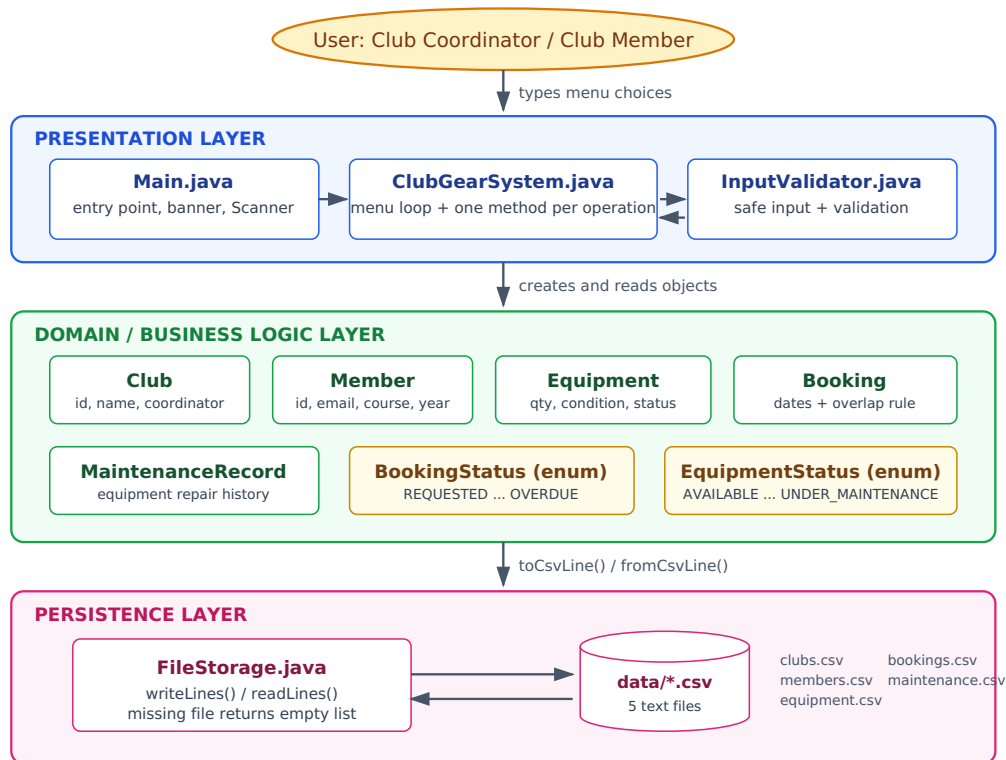


Figure 1 — Three-layer system architecture of ClubGear

Layer	Responsibility	Classes
Presentation	Show the menu, read input, print results and errors	Main, ClubGearSystem, InputValidator
Domain / business logic	Hold the data and the rules that belong to the data itself	Club, Member, Equipment, Booking, MaintenanceRecord, BookingStatus, EquipmentStatus
Persistence	Convert objects to text lines and back; read and write files	FileStorage and the toCsvLine() / fromCsvLine() methods on each model

5.1 Why this structure

- **The domain classes never print and never read the keyboard.** This is what makes `ValidationTests` possible: the overlap rule can be tested by calling `overlapsWith()` directly, with no simulated typing.
- **All the menu logic lives in one class.** `ClubGearSystem` holds the lists and one short method per menu option, so the path from a menu number to the code that runs is a single step.
- **All the input handling lives in one class.** Because `InputValidator` is the only place that touches the `Scanner`, the classic `nextInt()` / `nextLine()` bug cannot appear anywhere in the program.
- **Persistence is isolated.** Replacing CSV files with SQLite later would mean rewriting `FileStorage` and the conversion methods, and nothing else.

5.2 Class inventory

Class	Lines	Responsibility
Main	~30	Entry point: banner, creates the Scanner and the system, starts the menu
ClubGearSystem	~1000	All five data lists, the menu loop and one method per operation
Club	~90	Club record with CSV conversion
Member	~105	Member record with CSV conversion
Equipment	~155	Equipment record, status, <code>isBookable()</code> , quantity guard

Booking	~200	Booking record, <code>overlapsWith()</code> , <code>isBlockingEquipment()</code>
MaintenanceRecord	~85	One line of repair history
BookingStatus	~20	Enum of six booking states
EquipmentStatus	~18	Enum of four equipment states
FileStorage	~125	CSV read and write, missing-file handling
InputValidator	~155	Safe, re-prompting keyboard input and format validation
ValidationTests	~330	55 automated tests (in <code>tests/</code> , not part of the application)

6. Design Diagrams

6.1 Use Case Diagram

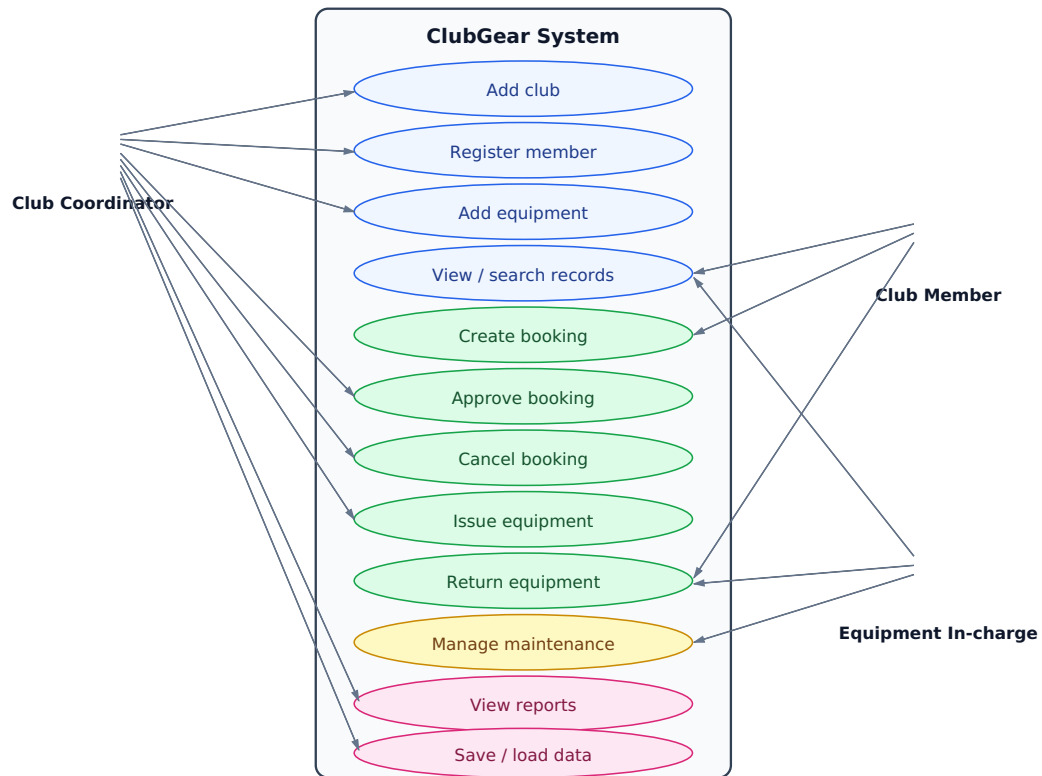


Figure 2 — Actors and their use cases

Use case	Actor	Pre-condition	Post-condition
Create booking	Member	Member, equipment and club exist; equipment is not damaged	A booking with status REQUESTED exists
Approve booking	Coordinator	Booking status is REQUESTED	Status becomes APPROVED
Issue equipment	Coordinator	Booking status is APPROVED	Booking ACTIVE, equipment BOOKED
Return equipment	Member / In-charge	Booking is ACTIVE or OVERDUE	Booking RETURNED, equipment released or flagged
Manage maintenance	In-charge	Equipment is DAMAGED or UNDER_MAINTENANCE	Equipment AVAILABLE, history entry written

6.2 Workflow Diagram — the booking life cycle



Only RETURNED and CANCELLED bookings release the equipment (business rule 10)

Figure 3 — Complete process flow of a booking, from request to return

6.3 Sequence Diagram — creating a booking

The booking creation flow is the most rule-heavy operation in the system, so it is the one shown in detail. Note steps 8 and 9 inside the loop: these are the calls that implement business rules 5 and 6.

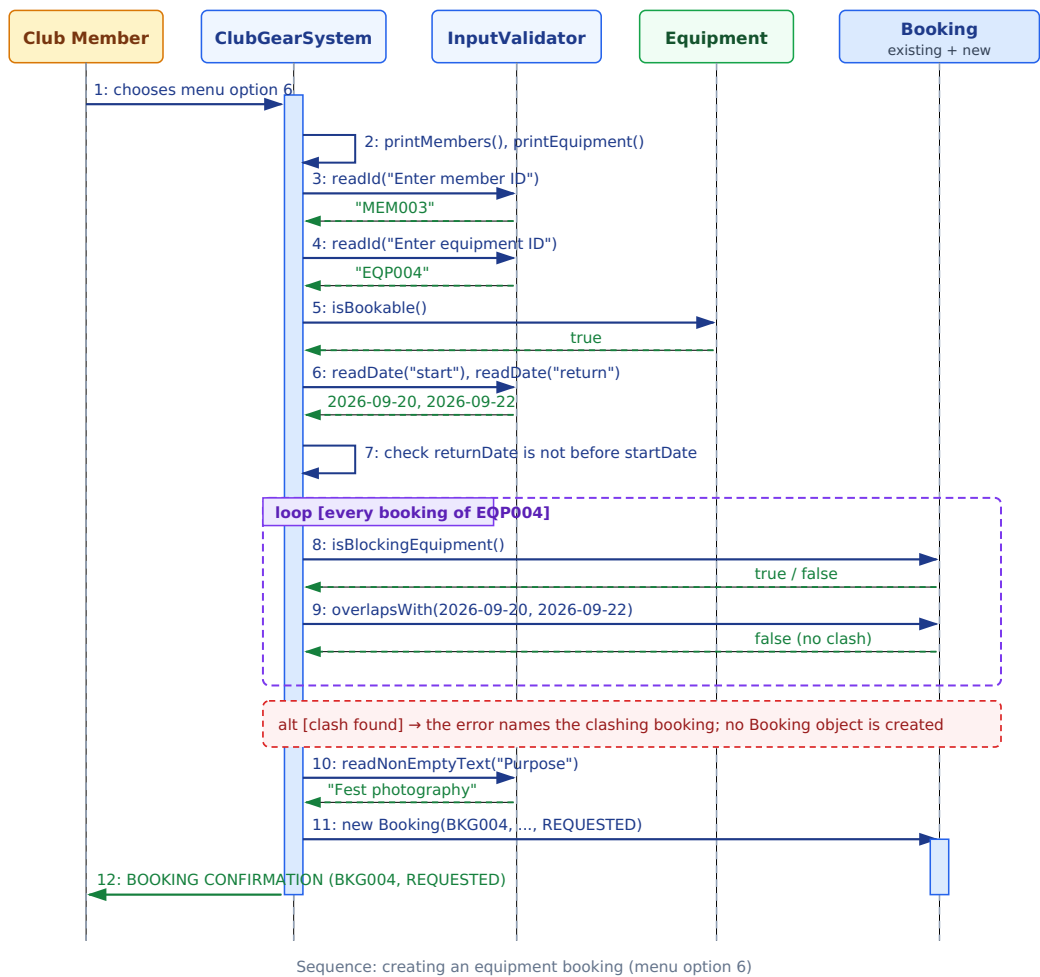


Figure 4 — Sequence of calls when a member creates a booking

6.4 Class Diagram

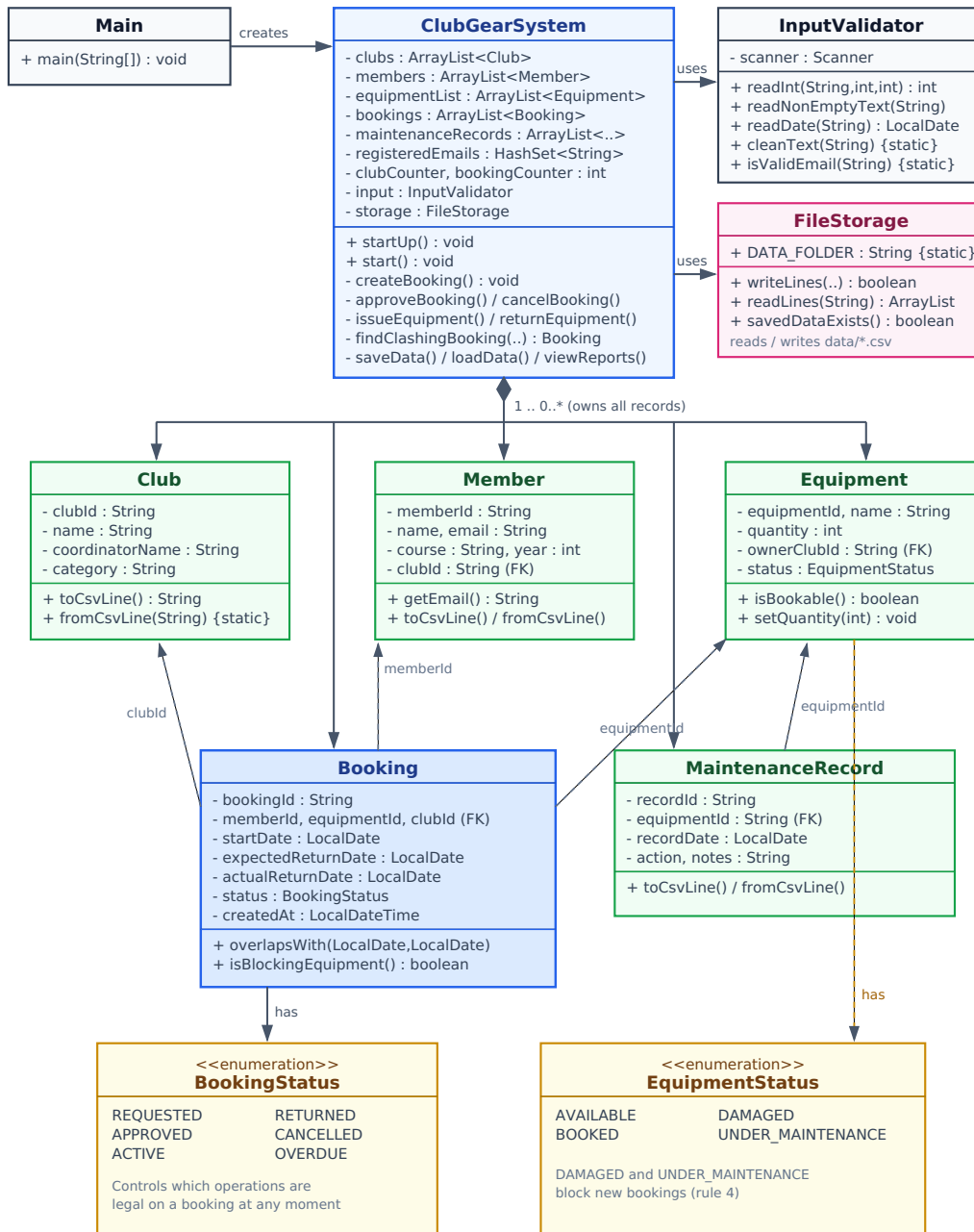
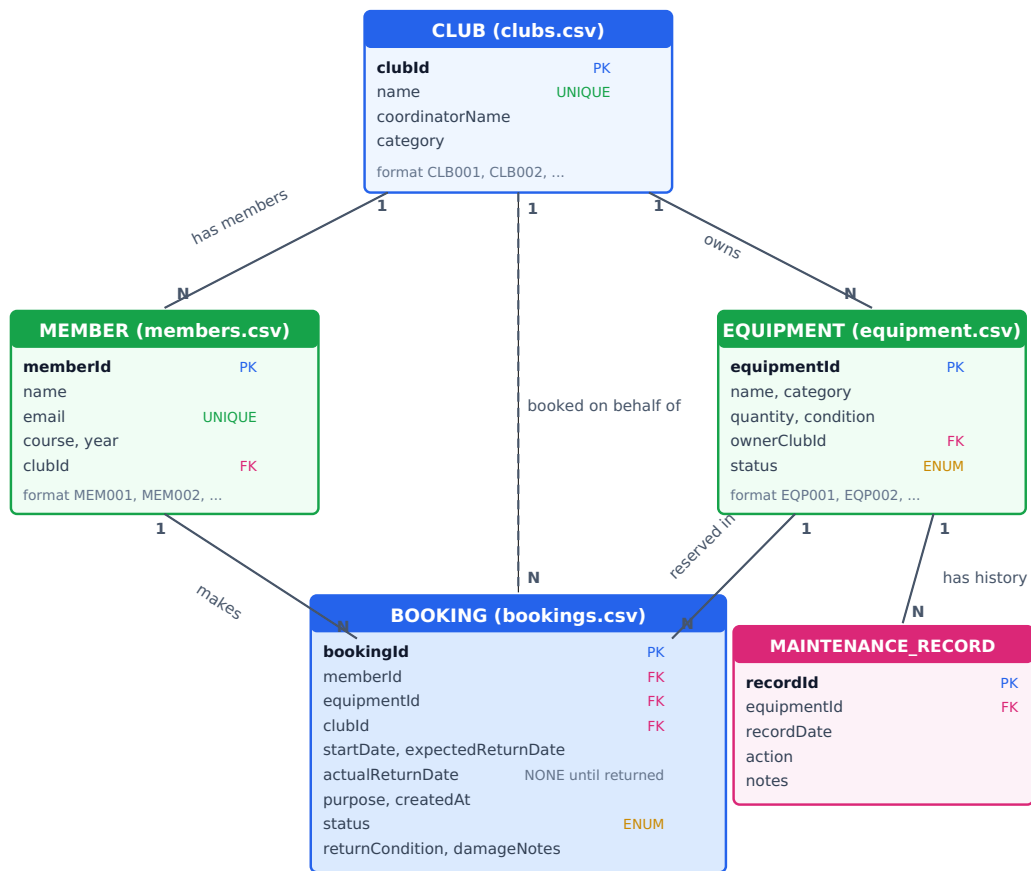


Figure 5 — Class diagram of the eleven application classes

6.5 ER Diagram and Storage Design

ClubGear has no database, but its storage is relational in structure: each CSV file behaves like a table, the first column is the primary key, and the ID columns act as foreign keys.



Each CSV file behaves like a table: column 1 is the primary key and the ID columns act as foreign keys

Figure 6 — Entity relationship view of the five CSV files

File schema

File	Columns
clubs.csv	clubId (PK), name (unique), coordinatorName, category
members.csv	memberId (PK), name, email (unique), course, year, clubId (FK)
equipment.csv	equipmentId (PK), name, category, quantity, condition, ownerClubId (FK), status (enum)
bookings.csv	bookingId (PK), memberId (FK), equipmentId (FK), clubId (FK), startDate, expectedReturnDate, actualReturnDate, purpose, status (enum), createdAt, returnCondition, damageNotes
maintenance.csv	recordId (PK), equipmentId (FK), recordDate, action, notes

Every file begins with a header line starting with #, which the loader skips. Empty optional fields are written as the word NONE. Real saved output:

```
# bookingId,memberId,equipmentId,clubId,startDate,expectedReturnDate,actualReturnDate,purpose,status,createdAt,returnCondition,damageNotes
BKG001,MEM001,EQP001,CLB001,2026-09-17,2026-09-19,NONE,Annual day photo coverage,APPROVED,2026-09-16T07:03:37.99,NONE,NONE
BKG002,MEM002,EQP002,CLB002,2026-09-10,2026-09-15,NONE,Tech talk audio setup,OVERDUE,2026-09-16T07:03:37.99,NONE,NONE
BKG003,MEM003,EQP003,CLB002,2026-08-27,2026-08-29,2026-08-29,Seminar presentation,RETURNED,2026-09-16T07:03:37.99,Good,NONE
```

7. Design Decisions and Rationale

7.1 No inheritance and no interfaces

The obvious temptation was a `Person` superclass for `Member`, or an `Entity` interface declaring `toCsvLine()`. Both were rejected. `Club`, `Member`, `Equipment` and `Booking` share no behaviour — only the coincidence that each has an ID — and inheritance built on a coincidence produces a base class that cannot be justified when questioned. The interface fails for a technical reason too: `fromCsvLine()` must be **static** because it creates the object, and a static factory cannot be part of an instance contract, so the interface would cover only half of the conversion and split related logic across two places.

7.2 One equipment record is one bookable asset

The alternative was a live stock count, where quantity 4 permits four simultaneous bookings. That requires tracking how many units are out on every overlapping date range — a genuinely harder algorithm than this syllabus calls for, and one that is easy to get subtly wrong. The chosen rule keeps `findClashingBooking()` to eight readable lines while still enforcing business rules 5 and 6 correctly. A club owning two cameras adds them as EQP001 and EQP002. The limitation is documented in the class comment, in the README and here, rather than hidden.

7.3 The overlap test is written as a negation

```
boolean noOverlap = otherEnd.isBefore(startDate) || otherStart.isAfter(expectedReturnDate);
return !noOverlap;
```

Testing for overlap directly needs four cases: the new range inside the old one, straddling its start, straddling its end, or surrounding it entirely. Testing for **non**-overlap needs only two, because two ranges fail to touch only when one finishes before the other begins. The negation is shorter and much easier to prove correct. All ten boundary cases — including the ranges that touch on exactly the first or last day — are covered by the automated tests.

7.4 OVERDUE bookings can still be returned

Business rule 9 says only active bookings may be returned. OVERDUE is not a different kind of booking; it is an ACTIVE booking that is late. Refusing a late return would leave the equipment permanently locked in the system, so `returnEquipment()` accepts ACTIVE and OVERDUE, and prints a note that the item came back after its due date.

7.5 Commas are stripped from user input

The data files use commas as separators, so a club named "Photography, Film and Media" would split a four-column line into five and silently corrupt the file at the next load. Writing a full quoting and escaping parser is real work and is not what the exercise is testing, so `InputValidator.cleanText()` replaces commas with spaces at the moment of entry. This is a trade-off, and it is stated as a limitation rather than presented as a feature.

7.6 Every input is read as a full line

Mixing `nextInt()` and `nextLine()` on one `Scanner` is the classic Java beginner bug: the newline left behind by `nextInt()` is consumed by the following `nextLine()`, which then returns an empty string. `InputValidator` reads every input with `nextLine()` and parses afterwards, so the bug cannot occur anywhere in the program.

7.7 Status is checked before it is changed

Each of `approveBooking()`, `cancelBooking()`, `issueEquipment()` and `returnEquipment()` begins by testing the current status and refusing with a message that names the actual state. This puts each transition of the state machine in exactly one place, instead of leaving the rules implied across the code.

7.8 IDs instead of object references

A `Booking` stores `memberId` and `equipmentId` as strings rather than holding `Member` and `Equipment` objects. This mirrors foreign keys in a database, makes each CSV line self-contained, and removes the need to re-link object graphs after loading a file. The cost is a lookup through `findMemberById()` when a name is needed for display, which is acceptable at this data size.

8. Implementation Details

8.1 Folder structure

```
ClubGear/
├── src/clubgear/
│   ├── Main.java
│   ├── ClubGearSystem.java
│   ├── Club.java
│   ├── Member.java
│   ├── Equipment.java
│   ├── InputValidator.java
│   ├── Booking.java
│   ├── MaintenanceRecord.java
│   ├── BookingStatus.java
│   ├── EquipmentStatus.java
│   └── FileStorage.java
├── tests/
│   ├── TestPlan.md (22 manual test cases)
│   └── clubgear/ValidationTests.java (55 automated tests)
├── docs/
│   ├── DESIGN.md
│   ├── SCREENSHOTS.md
│   ├── GIT_GUIDE.md
│   └── diagrams/*.svg
├── data/
│   └── (created at first save)
├── README.md
├── statement.md
└── .gitignore
```

8.2 Core Java concepts and where they are used

Concept	Where it does real work
Classes and objects	Five model classes, each representing one real-world record
Encapsulation	Every field is private; <code>setQuantity()</code> silently refuses values that are not positive, which is a rule the field itself owns
Constructors	Each model has a full constructor; <code>Booking</code> sets the three return fields to <code>null</code> because a new booking has not been returned yet
Enums	<code>BookingStatus</code> and <code>EquipmentStatus</code> make the state machine explicit and make an invalid state impossible to represent
<code>ArrayList</code>	The five record lists; also the filtered result lists returned by the search methods
<code>HashSet</code>	<code>registeredEmails</code> — an <code>O(1)</code> duplicate check instead of scanning every member
<code>HashMap</code>	Counting bookings per equipment and per club in the reports with <code>getOrDefault()</code>
Exception handling	<code>NumberFormatException</code> for typed numbers, <code>DateTimeParseException</code> for dates, <code>IOException</code> for file access, <code>IllegalArgumentException</code> for unknown enum words, <code>NoSuchElementException</code> for a closed input stream
<code>LocalDate</code>	Overlap comparison with <code>isBefore()</code> / <code>isAfter()</code> ; overdue detection against <code>LocalDate.now()</code> ; late days via <code>toEpochDay()</code>
<code>LocalDateTime</code>	The <code>createdAt</code> timestamp on each booking
Static methods	The <code>fromCsvLine()</code> factories and the two stateless helpers in <code>InputValidator</code>
File I/O	<code>BufferedWriter</code> / <code>BufferedReader</code> inside <code>try-with-resources</code> , so the stream closes even when an exception is thrown
Switch expressions	The arrow-form <code>switch</code> (Java 14+) for the menu, which removes the possibility of a fall-through bug

8.3 Key algorithm — clash detection

```
private Booking findClashingBooking(String equipmentId, LocalDate start, LocalDate end) {
    for (Booking booking : bookings) {
        if (booking.getEquipmentId().equals(equipmentId)
            && booking.isBlockingEquipment()
            && booking.overlapsWith(start, end)) {
            return booking;
        }
    }
    return null;
}
```

Three conditions in sequence: the same item, a booking that has not finished, and a date range that touches. Returning the clashing booking rather than a boolean is what allows the error message to name it: *"already booked from 2026-09-20 to 2026-09-22 (booking BKG004)"*.

8.4 Key algorithm — overdue detection

```
private void updateOverdueBookings() {
    LocalDate today = LocalDate.now();
    for (Booking booking : bookings) {
        if (booking.getStatus() == BookingStatus.ACTIVE
```

```

        && booking.getExpectedReturnDate().isBefore(today)) {
            booking.setStatus(BookingStatus.OVERDUE);
        }
    }
}

```

This runs at the top of every menu cycle and again before the reports are printed, so the overdue list is always current without any scheduler or timer.

8.5 Persistence cycle

1. **Save:** each list is converted to `ArrayList<String>` through `toCsvLine()`, then written with a header line by `FileStorage.writeLines()`.
2. **Load:** `readLines()` returns the non-comment lines; each is passed to the matching `fromCsvLine()`; a `null` result means a damaged line, which is counted and skipped.
3. **Counters:** `refreshCounters()` reads the numeric part of the highest existing ID so that the next generated ID continues the sequence instead of colliding with a loaded record.
4. **Clearing:** loading empties the lists first, so choosing Load Data twice cannot duplicate every record.

8.6 Sample data

When no `data` folder exists, the program loads two clubs, three members, five equipment items, three bookings and one maintenance record. The sample is designed for demonstration: one booking is APPROVED and ready to issue, one is issued and deliberately dated in the past so that overdue detection fires immediately, one is already returned so the reports have history, and one equipment item is DAMAGED so the rejection rule can be shown. Dates are generated relative to `LocalDate.now()`, so the demonstration works on any day the program is run.

9. Screenshots and Results

All output below was captured from a live run of the compiled program.

9.1 Start-up with no saved data

```
*****
*                  CLUBGEAR                  *
* Campus Club Equipment Booking and Management *
*****
No saved data found in the 'data' folder.
Loading built-in sample data for demonstration.
Sample data loaded: 2 clubs, 3 members, 5 equipment items, 3 bookings.
```

9.2 Main menu

```
=====
CLUBGEAR - MAIN MENU
Today's date: 2026-09-16
=====
1. Add Club                9. Cancel Booking
2. View / Search Clubs     10. Issue Equipment
3. Register / View Members 11. Return Equipment
4. Add Equipment           12. Manage Maintenance
5. View / Search / Update Equip 13. View Reports
6. Create Equipment Booking 14. Save Data
7. View Bookings           15. Load Data
8. Approve Booking         16. Exit
=====
Enter your choice (1-16):
```

(shown in two columns here to save space; the program prints one per line)

9.3 Equipment list

ID	NAME	CATEGORY	QTY	CONDITION	CLUB	STATUS
EQP001	Canon DSLR Camera	Camera	2	Good	CLB001	AVAILABLE
EQP002	Wireless Microphone	Audio	4	Good	CLB002	BOOKED
EQP003	Epson Projector	Display	1	Good	CLB002	AVAILABLE
EQP004	Camera Tripod	Camera	3	Average	CLB001	AVAILABLE
EQP005	LED Light Panel	Lighting	2	Poor	CLB001	DAMAGED
Total equipment shown: 5						

9.4 Successful booking

```
Enter member ID: MEM003
Enter equipment ID: EQP004
Booking for which club ID (usually the owner CLB001): CLB001
Booking start date (YYYY-MM-DD) : 2026-09-20
Expected return date (YYYY-MM-DD): 2026-09-22
Purpose of booking: Fest photography

===== BOOKING CONFIRMATION =====
Booking ID   : BKG004
Member      : Rahul Nair (MEM003)
Equipment   : Camera Tripod (EQP004)
Club        : Photography Club
From        : 2026-09-20
Until       : 2026-09-22
Purpose     : Fest photography
Status      : REQUESTED
=====
The booking is waiting for coordinator approval (menu option 8).
```

9.5 Conflict detection and rule enforcement

```
Error: this equipment is already booked from 2026-09-20 to 2026-09-22 (booking BKG004).

Error: this equipment is DAMAGED and cannot be booked right now.

Error: the return date cannot be earlier than the start date.

Error: booking BKG005 is CANCELLED. Only APPROVED bookings can be issued.

Error: a club named 'Photography Club' already exists.

Error: this email is already registered.
```

9.6 Approve, issue and return

```
Enter booking ID to approve: BKG004
Booking BKG004 approved. It can now be issued (menu option 10).

Enter booking ID to issue: BKG004
Equipment Camera Tripod issued to Rahul Nair.
Please return it on or before 2026-09-22.

Enter booking ID being returned: BKG004
Actual return date (YYYY-MM-DD): 2026-09-22
Condition after return (1 = Good, 2 = Damaged, 3 = Needs maintenance): 1
Equipment returned in good condition and is available again.
Booking BKG004 is now RETURNED.
```

9.7 Reports

```
=====
                        CLUBGEAR REPORTS
                Generated on 2026-09-16
=====
Total clubs           : 2
Total registered members : 3
Total equipment items  : 5
Available equipment    : 3
Currently booked equipment : 1
Damaged equipment      : 1
Equipment under maintenance: 0
Total bookings made    : 3
Completed (returned)   : 1
Cancelled bookings     : 0
Overdue bookings       : 1

--- Overdue bookings ---
BKG002 | Sneha Patil | Wireless Microphone | due 2026-09-15 | late by 1 day(s)

--- Most frequently booked equipment ---
EQP001 - Canon DSLR Camera : 1 booking(s)
EQP002 - Wireless Microphone : 1 booking(s)
EQP003 - Epson Projector : 1 booking(s)
Most booked item: Canon DSLR Camera (EQP001) with 1 booking(s).

--- Club-wise equipment usage ---
Photography Club (CLB001) : 1 booking(s), 3 item(s) owned
Coding Club (CLB002) : 2 booking(s), 2 item(s) owned

--- Equipment needing attention ---
EQP005 - LED Light Panel : DAMAGED
=====
```

9.8 Invalid input handled without crashing

```
Enter your choice (1-16): abc
That is not a valid number. Please try again.
Enter your choice (1-16): 99
Please enter a number between 1 and 16.

Quantity (1-500): 0
Please enter a number between 1 and 500.

Booking start date (YYYY-MM-DD) : 12-09-2026
Invalid date. Please use the format YYYY-MM-DD (example: 2026-09-16).
```

9.9 Saving and reloading

```
Data saved successfully inside the 'data' folder.
Clubs: 2, Members: 3, Equipment: 5, Bookings: 4, Maintenance records: 2

--- next run of the program ---
Saved data found. Loading it now...
Data loaded successfully.
Clubs: 2, Members: 3, Equipment: 5, Bookings: 4, Maintenance records: 2

--- with the data folder deleted ---
No saved data was found, so nothing was loaded.
(The current data in memory has not been changed.)
```

10. Testing Approach

Testing was done at two levels: an automated suite for the logic that can be checked without typing, and a written manual plan for the parts that need interaction.

10.1 Automated validation tests

`tests/clubgear/ValidationTests.java` contains a small hand-written harness — no JUnit, because the project is restricted to core Java. Each `check()` call prints PASS or FAIL, and the runner exits with code 0 only if everything passes, so it could be used in a CI script later.

Group	Tests	What is verified
Booking date overlap logic	10	Inside, straddling, surrounding, identical and touching ranges; the two non-overlapping cases one day either side
Booking status logic	7	Which of the six states block the equipment
Equipment status logic	5	<code>isBookable()</code> for all four states
Quantity rule	3	Positive accepted; zero and negative refused, old value kept
Email validation	7	Valid addresses; missing @, double @, no dot, leading @, trailing dot
Text cleaning	3	Commas removed, words preserved, spaces trimmed
CSV round trips	7	All five models survive save and load, including NONE handling for empty fields
Damaged CSV lines	6	Wrong column count, bad number, unknown enum word, bad date and empty line all return null instead of throwing
File storage	7	Missing file returns an empty list; write and read back; header skipped; temp file cleaned up
Total	55	All passing

```
$ java -cp out:out-tests clubgear.ValidationTests

[Booking date overlap logic]
PASS Fully inside the existing range overlaps
PASS Range touching the first day overlaps
PASS Range finishing one day earlier does NOT overlap
...
-----
Total tests : 55
Passed      : 55
Failed      : 0
-----
RESULT: ALL TESTS PASSED
```

10.2 Manual test plan

`tests/TestPlan.md` holds 22 manual test cases, each with a test case ID, description, input, expected output, and blank columns for actual output and result. They cover duplicate clubs, duplicate and malformed emails, invalid quantity, successful booking, overlapping and non-overlapping bookings, damaged equipment, invalid dates, invalid date format, approval, cancellation, issue, return in good condition, return with damage, overdue detection, maintenance, invalid menu input, save, reload and a missing data folder.

10.3 Results summary

Rule under test	Observed program behaviour
Duplicate club name	Error: a club named 'Photography Club' already exists.
Duplicate email	Error: this email is already registered.
Quantity of zero	Please enter a number between 1 and 500. then re-prompt
Overlapping booking	Refused, naming the clashing booking ID and its dates
Booking damaged equipment	Error: this equipment is DAMAGED and cannot be booked right now.
Return date before start date	Refused before the booking object is created
Issuing a cancelled booking	Error: booking BKG005 is CANCELLED. Only APPROVED bookings can be issued.

Overdue detection	BKG002 reported as late by 1 day in the reports
Approve → issue → return	Status chain REQUESTED → APPROVED → ACTIVE → RETURNED, equipment released
Maintenance cycle	UNDER_MAINTENANCE, then AVAILABLE after repair; both logged in the history
Invalid menu input	Re-prompt; program continues
Save and reload	Same record counts before and after restart
Missing data folder	Message printed, sample data loaded, no stack trace

10.4 Compilation check

```
$ javac --release 17 -Xlint:all -d out src/clubgear/*.java
(no output: zero errors, zero warnings)
```

11. Challenges Faced

11.1 Getting the date overlap rule right

The first attempt checked only whether the new start date fell inside an existing booking. That missed the case where a new booking completely surrounds an existing one — a five-day booking wrapping a two-day one would have been accepted. Rewriting the check as a negation ("they do not overlap only when one ends before the other starts") fixed the logic and reduced four conditions to two. The ten boundary tests in the automated suite exist because of this mistake.

11.2 The Scanner newline trap

Mixing `nextInt()` with `nextLine()` caused prompts to be skipped: the program appeared to ignore a question entirely and move on with an empty value. The cause is the newline character left in the buffer by `nextInt()`. The fix was structural rather than local — every read now goes through one `readLine()` method in `InputValidator`, so the situation cannot arise anywhere in the program.

11.3 Commas inside user text

Testing with the club name "Photography, Film and Media" corrupted `clubs.csv`: the line split into five columns, `fromCsvLine()` returned `null`, and the club vanished on the next load. A full CSV quoting parser was considered and rejected as out of scope, so commas are now removed at the point of entry and the restriction is documented as a known limitation.

11.4 Deciding what quantity means

An early version treated quantity as live stock, and the booking logic immediately became complicated: counting how many units were out required checking every overlapping booking and summing them per date. The scope decision — one record, one bookable asset — was taken deliberately and written down, because an examiner is likely to ask about it and a documented trade-off is defensible while a silent one is not.

11.5 Loading data twice

Choosing "Load Data" twice originally duplicated every record, because the loader appended to lists that already held data. Clearing all five lists and the email set at the start of `loadData()` fixed it. A second, related bug followed: after loading, the ID counters still started from 1, so the next new club would have been given an ID that already existed. `refreshCounters()` now scans the loaded IDs and continues from the highest one.

11.6 Keeping the scope honest

It was tempting to add a login system, an inheritance hierarchy and a fine calculator to make the project look more advanced. Each of those would have added code that could not be explained confidently in a viva. The decision was to keep the feature set focused and put the extras in the future enhancements list instead.

12. Learnings and Key Takeaways

1. **Enums prevent whole categories of bugs.** With `BookingStatus` as an enum, a typo like `"APROVED"` is a compile error rather than a silent runtime mismatch, and the compiler can check that every state is handled.
2. **Validation belongs at the boundary.** Putting every keyboard read in one class meant that the rest of the program could trust its inputs completely. No method in `ClubGearSystem` needs to re-check whether a number is really a number.
3. **Business rules should live in exactly one place.** The overlap rule is inside `Booking`, the bookability rule is inside `Equipment`. When a rule is written twice, the two copies eventually disagree.
4. **Negation can be simpler than enumeration.** The overlap check taught this directly: four positive cases became two negative ones, and the shorter version was easier to prove correct.
5. **Tests are most valuable at the boundaries.** The bugs were never in the middle of a date range; they were on the exact day where two bookings touch. The tests that found real problems were the edge cases.
6. **File handling needs a plan for bad data.** Deciding early that `fromCsvLine()` returns `null` for a damaged line — rather than throwing — made the whole load path simple and crash-proof.
7. **A documented limitation is stronger than a hidden one.** Writing down why quantity is not a live stock count turns a gap into a design decision that can be discussed.
8. **Not using a feature is also a design decision.** Choosing not to use inheritance, and being able to explain why, demonstrates more understanding than adding a superclass that serves no purpose.
9. **try-with-resources should be the default.** The file streams close themselves even when an exception is thrown, which removes an entire class of resource leak without any extra code.
10. **Small methods make debugging fast.** Because each menu option maps to one short method, every bug during development was traceable to a single method within a minute.

Overall takeaway. The hard part of this project was not writing Java syntax; it was deciding what the rules should be and where each rule belongs. Once the twelve business rules were written down explicitly, the code almost arranged itself.

13. Future Enhancements

#	Enhancement	Detail
1	Role-based login	Separate coordinator and member accounts so that only a coordinator can approve bookings; passwords stored as hashes, not plain text
2	Real stock counting	Allow 3 of 4 microphones to be out at once by tracking units booked per date range
3	Return reminders	Email or SMS the day before a booking is due, using the JavaMail API
4	Late fines	Calculate a fine per late day and add it to a member's record
5	Report export	Write the reports to a text or CSV file that can be printed or shared
6	Database storage	Replace CSV files with SQLite through JDBC; only <code>FileStorage</code> and the conversion methods would change
7	Graphical interface	A JavaFX or Swing front end built on the existing domain classes, which need no modification
8	QR codes	A QR sticker on each item so issue and return can be done by scanning instead of typing an ID
9	Booking history per member	A reliability score based on how often a member returns items on time
10	Multi-club sharing rules	Let one club lend to another with an approval step from both coordinators

14. References

1. Oracle. *Java Platform, Standard Edition 17 API Specification*. Oracle Corporation. Packages used: `java.util`, `java.time`, `java.io`. <https://docs.oracle.com/en/java/javase/17/docs/api/>
2. Oracle. *The Java Tutorials — Collections Trail*. Reference for `ArrayList`, `HashMap` and `HashSet` behaviour. <https://docs.oracle.com/javase/tutorial/collections/>
3. Oracle. *The Java Tutorials — Date Time Trail*. Reference for `LocalDate`, `LocalDateTime` and ISO-8601 parsing. <https://docs.oracle.com/javase/tutorial/datetime/>
4. Oracle. *The Java Tutorials — Exceptions*. Reference for try-with-resources and multi-catch. <https://docs.oracle.com/javase/tutorial/essential/exceptions/>
5. Herbert Schildt. *Java: The Complete Reference*, 12th edition. McGraw Hill. Chapters on classes, inheritance, exceptions, enumerations and I/O.
6. Kathy Sierra and Bert Bates. *Head First Java*, 3rd edition. O'Reilly Media. Reference for object-oriented design reasoning.
7. Robert C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall. Referred to for naming, method length and the single responsibility idea.
8. Object Management Group. *Unified Modeling Language Specification, version 2.5.1*. Notation used for the use case, class and sequence diagrams in section 6. <https://www.omg.org/spec/UML/>
9. Y. Shafranovich. *RFC 4180 — Common Format and MIME Type for Comma-Separated Values (CSV) Files*. Internet Engineering Task Force, 2005. Consulted when deciding how to handle commas inside field values.
10. VITYarthi. *Build Your Own Project — General Project Instructions and Submission Guidelines*. Course handout followed for the structure of this report.

— End of report —

